

PRD — Dynamic Quote Builder (Google Sheets Powered)

Product Requirements Document

1. Product Overview

The Dynamic Quote Builder is a SaaS tool for **interior designers and architects** to rapidly create **accurate, material-driven, unit-driven, room-driven quotations**.

All pricing, materials, room type rules, unit types, finishes, and markup logic are maintained in a single Google Sheets workbook, making it easy for the business team to update costing without developers.

The system must:

- Read data **directly from Google Sheets**
 - Filter line items dynamically based on category, room type, unit type
 - Allow architects to input dimensions
 - Auto-calculate sqft, mm values, costs, markup, GST
 - Generate a structured quotation summary
 - Produce a downloadable PDF (future phase)
 - Provide a clean UI for architects
 - Provide a restricted admin UI for the owners (via the Sheet)
-

2. High-Level Architecture

Front-End

- React (via Replit template)
- Typescript recommended
- API calls to backend for price catalog + calculations
- Client-side caching to minimize sheet hits

Backend

- Node.js (Express) or Python FastAPI
- Server functions:
 - Fetch + parse Google Sheets
 - Cache pricing data in memory
 - Provide filtered lists: categories → room types → unit types → items
 - Compute line-item totals
 - Compute full quotation totals

Data Source: Google Sheets

Google Sheet: **“Quote Gen - Blueprint”**

Tabs:

- Economy
- Lite Premium (Mid Premium)
- Premium
- Luxury

Sheet columns (normalized):

Project Type | Section | Room Type | Unit Type | Panel Type | Category | Material Type | Brand | Description | Rate | Markup | Markup Value | Selling Price

State Management

- Per project:
 - selected category
 - rooms
 - line items
 - architect-entered values

Authentication

- JWT-based simple auth
 - Roles:
 - Architect
 - Admin (sheet-only, backend should block schema changes)
-

3. Functional Requirements

3.1 Project Management

Architect can:

- Create a new project
 - Enter:
 - Client Name
 - Project Type (Residential / Commercial / Others)
 - Style Category (Economy / Mid / Premium / Luxury)
 - Toggle: "Multiple Style Preferences"
 - After creation, project opens in "Quotation Builder" view.
-

3.2 Material Specification (Auto-Generated)

Upon project creation:

- System must read **unique** material specs from the chosen sheet:
 - Core materials
 - Finish materials
 - Hinges
 - Other specs (brand, laminate type etc.)
- Display as a formatted block above the quote builder.

If "Multiple Style Preferences" is ON:

- Each room inherits its own category
 - Material spec updates per room
-

3.3 Add Space (Room)

Architect adds a room with:

- Room Type
 - Wet / Exposed
 - Dry / Inexposed
- Room Name (text)
- Unit Group Name (optional)
- If multiple-style enabled → Select Category

System creates a Room record:

room_id
project_id
room_type
room_name
unit_group_name (optional)
pricing_category (inherited or selected)

3.4 Add Line Item (Core Logic)

Inside each room:

User clicks **Add Line Item**.

Step 1: Select Unit Type

Dropdown or searchable list:

- Base Storage
- Wall Storage
- Tall Storage
- Wardrobe

Step 2: Select Description (as per sheet)

System filters items from Google Sheets by:

1. Pricing category (sheet name)
2. Room Type (Wet/Dry)
3. Unit Type
4. Section = "Modular"
5. Rate must be numeric

Display description text from Column I.

Step 3: Architect Enters Dimensions

- Length (feet)
- Height (feet)
- Quantity (default 1)

Step 4: System Auto-Calculates

- $\text{sqft} = \text{length} * \text{height}$
- $\text{length_mm} = \text{length} * 300$
- $\text{height_mm} = \text{height} * 300$
- $\text{rate} = \text{selling_price}$ (Column M)
- $\text{amount} = \text{rate} * \text{sqft} * \text{quantity}$

Save as `QuoteLineItem`.

3.5 Quotation Summary

System calculates:

Subtotal

```
sum(line_item.amount)
```

Markup %

Architect can enter optional markup at project level.

```
markup_value = subtotal * (markup_pct / 100)
```

Total After Markup

```
total_with_markup = subtotal + markup_value
```

GST (18%)

```
gst_amount = total_with_markup * 0.18
```

Grand Total

```
grand_total = total_with_markup + gst_amount
```

Display in summary panel.

3.6 Google Sheets Integration Requirements

Polling / Fetch Strategy

- Cache sheet data for 5–10 minutes
- Manual “refresh catalog” button (admin only)
- Never write to sheet from app

Normalisation Layer

Backend must normalize sheet into:

```
[
  {
    id,
    category_name,
    room_type,
    unit_type,
    material_type,
    description,
    rate,
    markup_pct,
    selling_price
  },
  { ... }
]
```

Protection

- Only values changeable by admin are:
 - Rate
 - Markup
 - Description (optional)
- App must validate that schema (column structure) is intact.

4. Non-Functional Requirements

Scalability

- <20 architects initially
- Max safe load: 200–500 daily quote calculations
- Caching prevents API overuse

Reliability

All calculations must **mirror Excel exactly**.

Performance

- Rendering quote screen < 200ms after API call
- Dropdown search < 100ms results

Security

- JWT auth
 - Architect cannot see rate/markup columns directly
 - Admin edits only through Google Sheets
-

5. UI/UX Requirements

5.1 Project List

Columns:

- Client Name
- Status (Draft / Generated)
- Total Quotes
- Last Updated

5.2 Project Detail

Sections:

- Material Specification block
- Add Space button
- Room list
- Quotation Summary panel (fixed on right side)

5.3 Room Detail

Columns:

- Description
- Length (ft)
- Height (ft)
- Sqft
- mm conversions
- Rate
- Quantity
- Amount

Bottom:

- Add Line Item button
-

6. Backend API Specification

GET /api/catalog

Returns normalized price items.

GET /api/catalog/:category

Returns items filtered by category.

POST /api/project

Creates new project.

POST /api/project/:id/room

Creates a room.

POST /api/room/:id/line-item

Creates a line item.

GET /api/project/:id/quote

Returns all line items + totals.

7. Data Models

Project

project_id
client_name
project_type
default_pricing_category
multi_style_enabled
created_by
created_at

Room

room_id
project_id
room_name
room_type
unit_group_name
pricing_category

LineItem

item_id
room_id

project_id
description
unit_type
rate
length_ft
height_ft
sqft
length_mm
height_mm
quantity
amount

CatalogItem

(normalized Google sheet rows)

catalog_id
category_name
room_type
unit_type
description
rate
markup_pct
selling_price
material_type

8. Error Handling Requirements

- Sheet unavailable → return cached version + warning
 - Invalid sheet structure → block sync + notify admin
 - Rate or markup is non-numeric → skip row
 - Architect enters invalid numbers → inline validation
-